

THE UNIVERSITY OF AZAD JAMMU AND KASHMIR, MUZAFFARABAD



LAB 02

FROM REAL WORLD CONCEPT TO SOFTWARE OBJECT (HOSPITAL MANAGEMENT SYSTEM)

SUBMISSION DETAILS

Student Name	Niqash Ali
Roll Number	2025-SE-34
Course Code	SEC-532
Course Name	Software Design & Architecture
Instructor	Engr. Ghulam Rabbani Butt
Semester	3rd
Session	2025–29
Lab Title	From Real World Concept to Software Object (Hospital Management System)
Assigned On	October 21, 2026
Due Date	October 28, 2026
Submission Date	October 26, 2026
University	The University of Azad Jammu and Kashmir

Hospital Management System: Conceptual Classes and Abstraction

1. Introduction and Selected Domain

For this assignment I selected the Hospital Management System as my real-world domain. A hospital manages many activities at the same time, such as patient registration, doctors, appointments, medical records, admissions, rooms, prescriptions, laboratory tests and payments. When these activities are handled without a proper structure, information can be misplaced and patients may experience delays. The purpose of this system is to organize hospital information in one place so that daily hospital operations become easier to manage.

2. Candidate Conceptual Classes

I identified the following 10 conceptual classes. They are conceptual classes because each one represents a meaningful entity or idea in the hospital domain, rather than a screen, a database table or a piece of code.

1. **Patient:** A person receiving care at the hospital. It stores identification and contact details.
2. **Doctor:** A medical professional who examines patients and provides treatment.
3. **Appointment:** A planned visit between a patient and a doctor, including its date and status.
4. **MedicalRecord:** A record of a patient's diagnoses, treatment notes and medical history.
5. **Department:** A hospital unit such as Cardiology, Pediatrics or Emergency.
6. **Nurse:** A healthcare worker who supports patient care and follows assigned duties.
7. **Room:** A hospital room or bed space, including its type and availability.
8. **Admission:** A record of a patient's hospital stay, including admission and discharge dates.
9. **Prescription:** A record of medicines and instructions prescribed for a patient.
10. **Bill:** A record of charges for consultations, tests, room use and other hospital services.

3. Attributes and Methods of Three Important Classes

Attributes are the pieces of information a class stores, while methods are the actions or behaviors the class can perform. I chose Patient, Doctor and Appointment for detailed modeling.

3.1 Patient Class

Attributes

- **patientId:** A unique identifier used to distinguish one patient from another.
- **fullName:** Stores the patient's complete name.
- **dateOfBirth:** Records the patient's date of birth.
- **phoneNumber:** Stores the patient's contact number.
- **address:** Stores the patient's home address.

Methods

- **registerPatient():** Creates a new patient record.
- **updateContactDetails():** Updates the patient's phone number or address.
- **viewMedicalHistory():** Displays the patient's recorded medical history.
- **bookAppointment():** Requests an appointment with a doctor.

3.2 Doctor Class

Attributes

- **doctorId:** A unique identifier for each doctor.
- **fullName:** Stores the doctor's complete name.
- **specialty:** Identifies the doctor's medical field.
- **phoneNumber:** Stores the doctor's contact number.
- **availability:** Shows the doctor's available days or time slots.

Methods

- **viewAppointments():** Displays the doctor's scheduled appointments.
- **updateAvailability():** Changes the doctor's available time slots.
- **addTreatmentNotes():** Adds notes about a patient's treatment.
- **reviewMedicalRecord():** Opens the relevant patient medical record for review.

3.3 Appointment Class

Attributes

- **appointmentId:** A unique identifier for each appointment.
- **patientId:** Identifies the patient who booked the appointment.
- **doctorId:** Identifies the doctor for the appointment.
- **appointmentDate:** Records the appointment date.
- **appointmentTime:** Stores the scheduled time.
- **status:** Shows whether the appointment is booked, completed or cancelled.

Methods

- **bookAppointment():** Creates a new appointment.
- **rescheduleAppointment():** Changes the date or time of an appointment.
- **cancelAppointment():** Cancels an appointment.
- **updateStatus():** Changes the current appointment status.

4. Main Abstraction Decisions

Patient, Doctor and Appointment were selected for detailed modeling because they are central to the daily work of a hospital. The other classes support patient care, medical services, room management and billing. Advanced pharmacy stock control, staff payroll, insurance claim processing, ambulance tracking and detailed laboratory equipment management were deliberately excluded because they are beyond the scope of this small model. Screens, database tables, login controllers and programming language details were also left out, since conceptual modeling focuses on the real-world hospital domain rather than technical implementation.

5. Invented Software Object

Appointment Reminder Scheduler

An Appointment Reminder Scheduler is an invented software object. It does not represent a physical entity in the hospital, but it is useful in the system design.

- **Purpose:** Sends a reminder to a patient before a scheduled appointment.
- **Scheduling:** Checks the appointment date and chooses an appropriate time to send the reminder.
- **Result:** Records whether the reminder was sent successfully or failed.
- **Benefit:** Helps patients remember their appointments and can reduce missed visits.

Even though it has no direct physical counterpart in the hospital, this object shows that software design sometimes needs objects created to coordinate activities or apply system rules.

6. Conclusion

The Hospital Management System is a practical example of conceptual modeling. By identifying the main classes, defining their attributes and methods, and making clear abstraction decisions, we can organize the system around real hospital needs before thinking about implementation. Keeping each class focused makes the design easier to understand, maintain and improve.